МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение

высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 13

Тема: Поведенческие паттерны: Паттерны микросервисов и распределенных систем

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы паттернов микросервисов «API Gateway» и «Circuit Breaker» и реализовать прикладную задачу «Графический пароль» с использованием данных паттернов.

Задание: необходимо разработать приложение для работы с графическим паролем и реализовать один из паттернов микросервисов: Saga, CQRS, Event Sourcing, Circuit Breaker, API Gateway.

В соответствии с заданием были выбраны паттерны «API Gateway» и «Circuit Breaker».

Вариант 8 - Графический пароль

Разработанное приложение позволяет формировать, сохранять и проверять графический пароль. Все обращения к «сервисам» проходят через единый шлюз (API Gateway), а отказоустойчивость обеспечивается предохранителем (Circuit Breaker).

1. Описание паттерна «API Gateway»

Паттерн API Gateway - это архитектурный паттерн микросервисных систем, при котором все клиентские запросы проходят через единую точку входа. Шлюз маршрутизирует запросы к соответствующим микросервисам, выполняет аутентификацию, логирование и другие сквозные задачи.

Данный паттерн применяется, когда клиенту не нужно знать о внутренней структуре микросервисов, а также когда требуется централизованная обработка запросов.

В разработанном приложении паттерн API Gateway используется следующим образом:

- APIGateway - единая точка входа;
- ValidatorService - микросервис валидации (POST /validate);
- PasswordService - микросервис хранения (POST /password,POST /login, DELETE /password, GET /status).

  Методы шлюза:

  def validate(self, sequence):

```
        return self._call("POST /validate", lambda:
    self.validator_service.validate(sequence))

    def create(self, sequence):
        return self._call("POST /password", lambda:
    self.password_service.create(sequence))

    def verify(self, sequence):
        return self._call("POST /login", lambda:
    self.password_service.verify(sequence))
```

Клиент (AppWindow) работает только с APIGateway и не знает, какие сервисы стоят за эндпоинтами.

2. Описание паттерна «Circuit Breaker»

Паттерн Circuit Breaker - это паттерн отказоустойчивости, который предотвращает повторные вызовы сбойного сервиса. Он работает по аналогии с электрическим предохранителем: после определённого числа ошибок «размыкает цепь» и блокирует запросы на заданное время.

Состояния:

- CLOSED - запросы проходят;
- OPEN - запросы блокируются на T секунд;
- HALF-OPEN - пробный запрос (в упрощённой реализации - сразу CLOSED после успеха).

В разработанном приложении реализован упрощённый Circuit Breaker:

- failure_threshold = 3 - после 3 ошибок цепь открывается;
- recovery_time = 10 - через 10 секунд цепь закрывается;

При каждом вызове APIGateway проверяет breaker.is_open():

- если открыт - возвращает ServiceResult(False, "Circuit Breaker открыт...");
- иначе выполняет запрос, и при ошибке вызывает record_failure(), при успехе - record_success().

Состояние отображается над сеткой: CLOSED (зелёный) или OPEN (N с) (красный). Кнопка «Симулировать сбой» позволяет вручную увеличить счётчик ошибок для демонстрации.

3. Описание разработанного приложения

Разработанное приложение предназначено для формирования, сохранения и проверки графического пароля.

- Основное окно содержит:
- бейдж состояния Circuit Breaker;
- сетку 3×3 из девяти скруглённых ячеек;
- текущую последовательность;
- основную кнопку («Войти» / «Сохранить пароль»);
- кнопку «Очистить»;
- переключатель режимов «Вход» / «Создать пароль»;
- кнопки «Лог Gateway» и «Симулировать сбой»;
- кнопку «Сбросить сохранённый пароль»;
- статусную строку.

К паролю предъявляются требования:

- длина от 4 до 9 ячеек;
- отсутствие повторов;
- ячейки в диапазоне 1–9.

4. Диаграмма классов

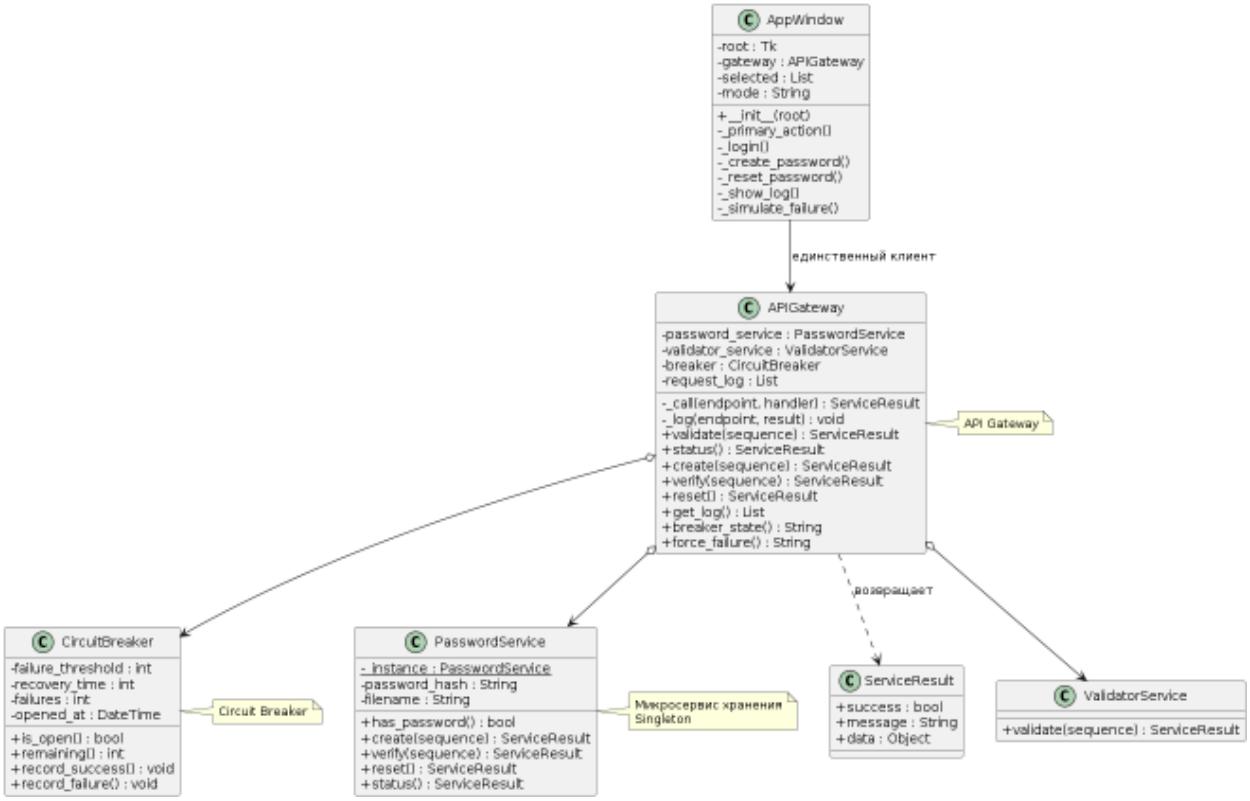


Рисунок 1 - Диаграмма классов приложения «Графический пароль»

Диаграмма содержит шлюз, два «микросервиса», предохранитель и UI-клиент. AppWindow связан только с APIGateway, что соответствует паттерну API Gateway.

5. Реализация паттерна «API Gateway»

```
class APIGateway:
    def __init__(self):
        self.password_service = PasswordService()
        self.validator_service = ValidatorService()
        self.breaker = CircuitBreaker(failure_threshold=3, recovery_time=10)
        self.request_log = []

    def _log(self, endpoint, result):
        timestamp = datetime.now().strftime("%H:%M:%S")
        self.request_log.append((timestamp, endpoint, result.success,
result.message))
```

```
        if len(self.request_log) > 20:
            self.request_log.pop(0)

    def _call(self, endpoint, handler):
        if self.breaker.is_open():
            remaining = self.breaker.remaining()
            result = ServiceResult(False, f"Circuit Breaker открыт. Подождите
{remaining} с.")
            self._log(endpoint, result)
            return result
        try:
            result = handler()
            if not result.success:
                self.breaker.record_failure()
            else:
                self.breaker.record_success()
            self._log(endpoint, result)
            return result
        except Exception as e:
            self.breaker.record_failure()
            result = ServiceResult(False, f"Ошибка сервиса: {e}")
            self._log(endpoint, result)
            return result

    def validate(self, sequence):
        return self._call("POST /validate", lambda:
self.validator_service.validate(sequence))

    def status(self):
        return self._call("GET /status", lambda: self.password_service.status())
```

```
    def create(self, sequence):
        return self._call("POST /password", lambda:
self.password_service.create(sequence))

    def verify(self, sequence):
        return self._call("POST /login", lambda:
self.password_service.verify(sequence))

    def reset(self):
        return self._call("DELETE /password", lambda:
self.password_service.reset())
```

Метод _call - центральное место шлюза. Он проверяет состояние Circuit Breaker, вызывает нужный сервис через переданную лямбду, фиксирует успех или ошибку в предохранителе и записывает обращение в журнал. Благодаря этому UI (AppWindow) никогда не обращается к PasswordService или ValidatorService напрямую - только через APIGateway.

6. Реализация паттерна «Circuit Breaker»

```
class CircuitBreaker:
    def __init__(self, failure_threshold=3, recovery_time=10):
        self.failure_threshold = failure_threshold
        self.recovery_time = recovery_time
        self.failures = 0
        self.opened_at = None

    def is_open(self):
        if self.opened_at is None:
            return False
```

```
            if datetime.now() >= self.opened_at +
        timedelta(seconds=self.recovery_time):
                self.opened_at = None
                self.failures = 0
                return False
            return True

        def remaining(self):
            if self.opened_at is None:
                return 0
            delta = (self.opened_at + timedelta(seconds=self.recovery_time)) -
        datetime.now()
            return max(0, int(delta.total_seconds()))

        def record_success(self):
            self.failures = 0
            self.opened_at = None

        def record_failure(self):
            self.failures += 1
            if self.failures >= self.failure_threshold:
                self.opened_at = datetime.now()
```

Переходы состояний:

Текущее состояние	Событие	Новое состояние
CLOSED	успешный вызов	CLOSED (сброс счётчика)
CLOSED	ошибка (счётчик < 3)	CLOSED
CLOSED	ошибка (счётчик ≥ 3)	OPEN
OPEN	запрос от клиента	OPEN (возврат ошибки)
OPEN	прошло T секунд	CLOSED (сброс)

В упрощённой реализации состояние HALF-OPEN не выделено отдельно: после истечения recovery_time предохранитель сразу переходит в CLOSED, и следующий запрос проверяет сервис заново.

7. Взаимодействие компонентов

Поток выполнения запроса «Создать пароль»:

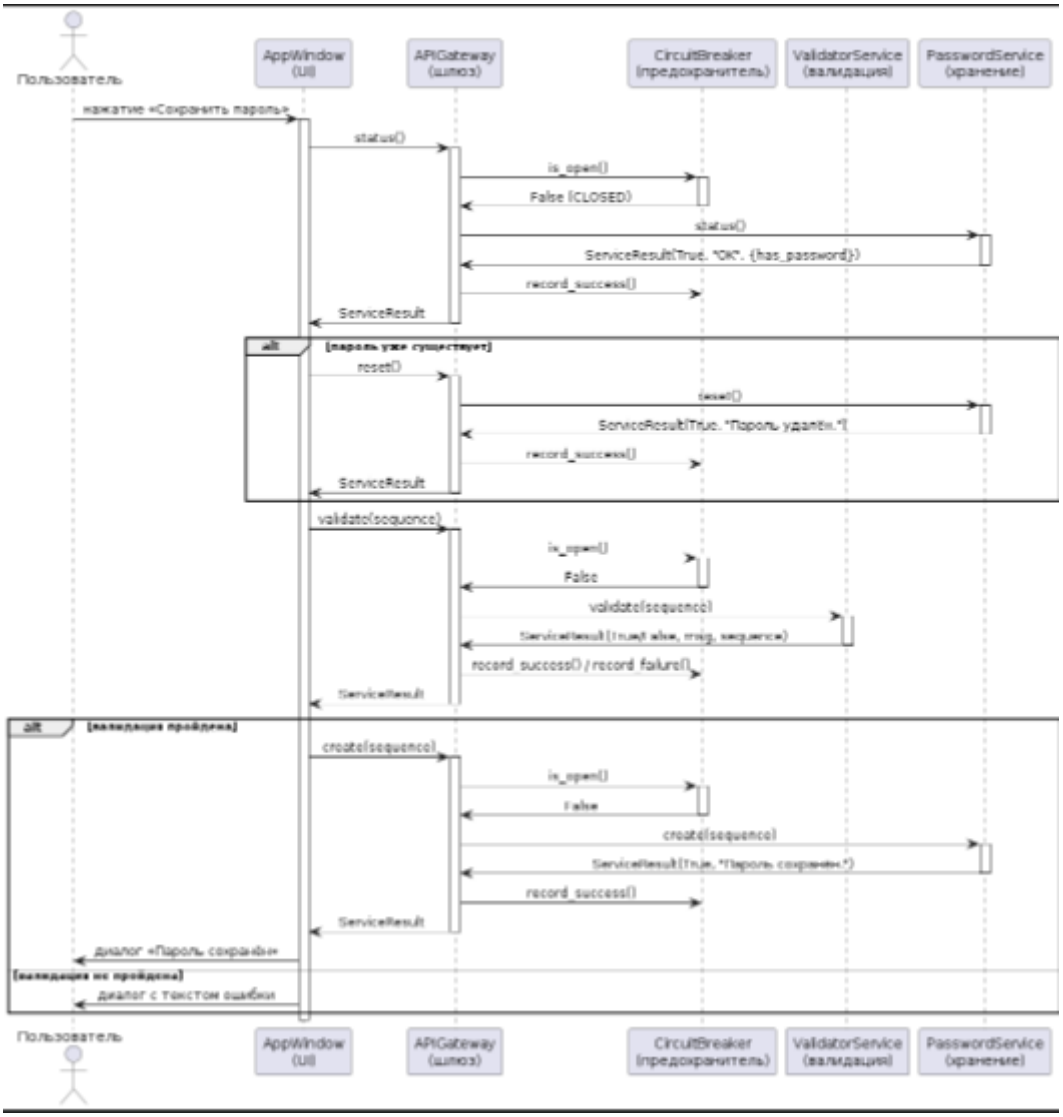


Рисунок 2 - Диаграмма последовательности для сценария «Создать пароль»

UI обновляется по ServiceResult

Каждый шаг проходит через _call, поэтому Circuit Breaker «видит» все обращения - как успешные, так и неуспешные. Это ключевое преимущество: предохранитель защищает весь шлюз, а не отдельный метод.

8. Результаты работы программы

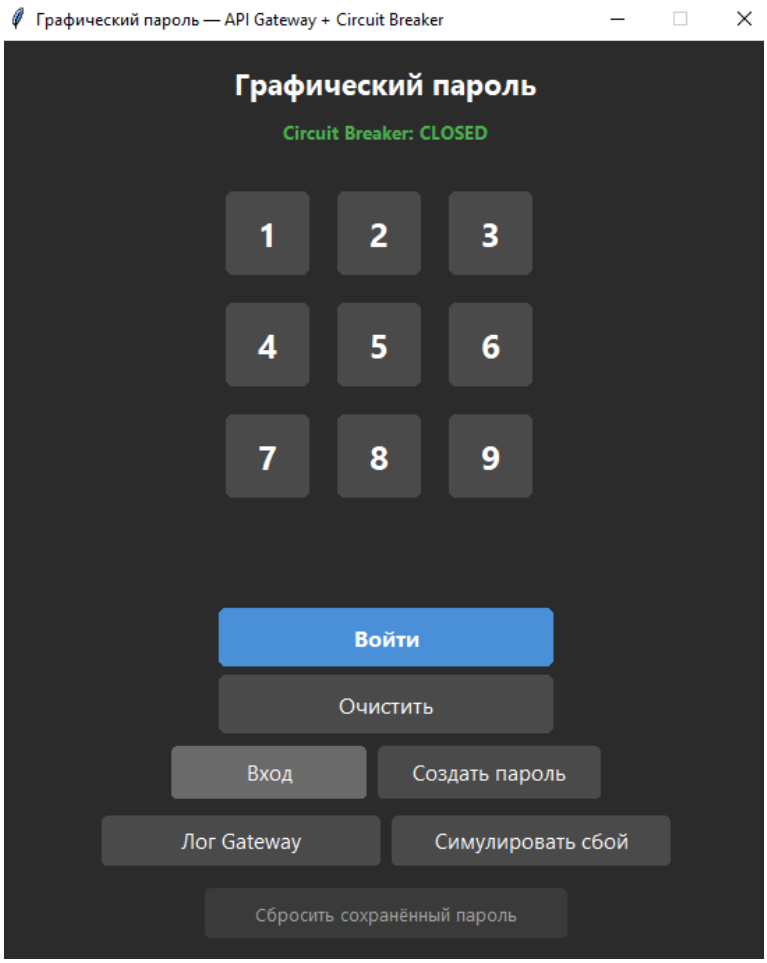


Рисунок 3 - Главное окно приложения.

Отображается заголовок «Графический пароль», бейдж Circuit Breaker:

– CLOSED (зелёный), сетка 3×3 из скруглённых ячеек, кнопки «Войти»,
– «Очистить», переключатель «Вход / Создать пароль», кнопки «Лог Gateway»,
– «Симулировать сбой» и «Сбросить сохранённый пароль».

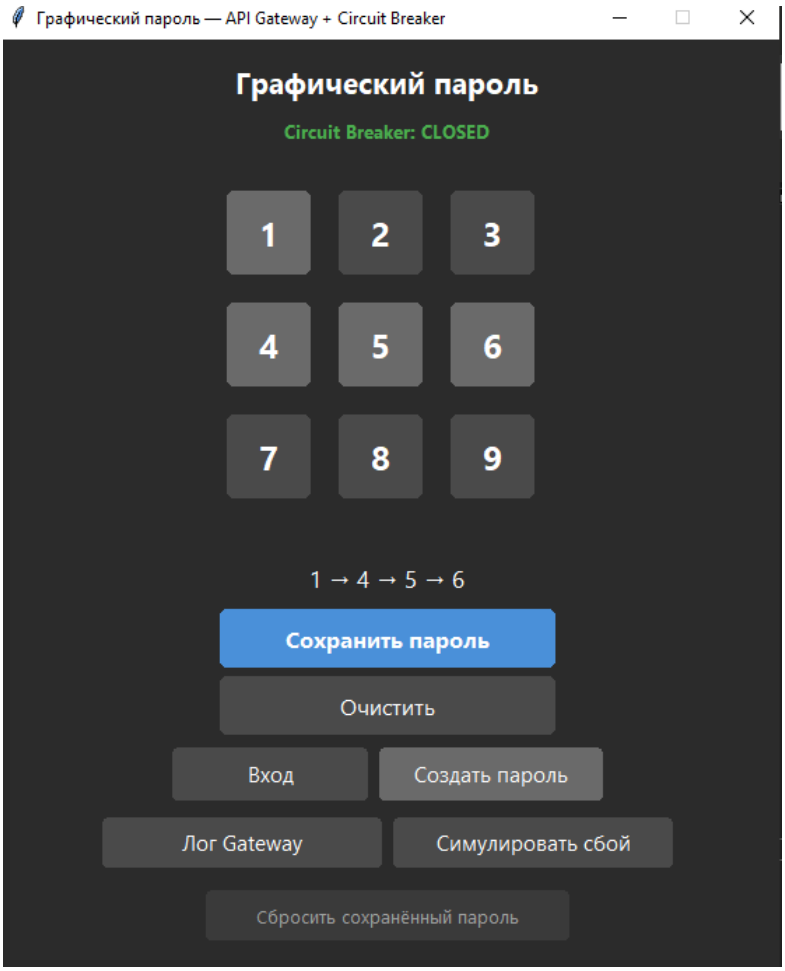


Рисунок 4 - Формирование последовательности.

Пользователь выбрал ячейки 1 → 4 → 5 → 6; они подсвечены, строка под сеткой показывает последовательность.

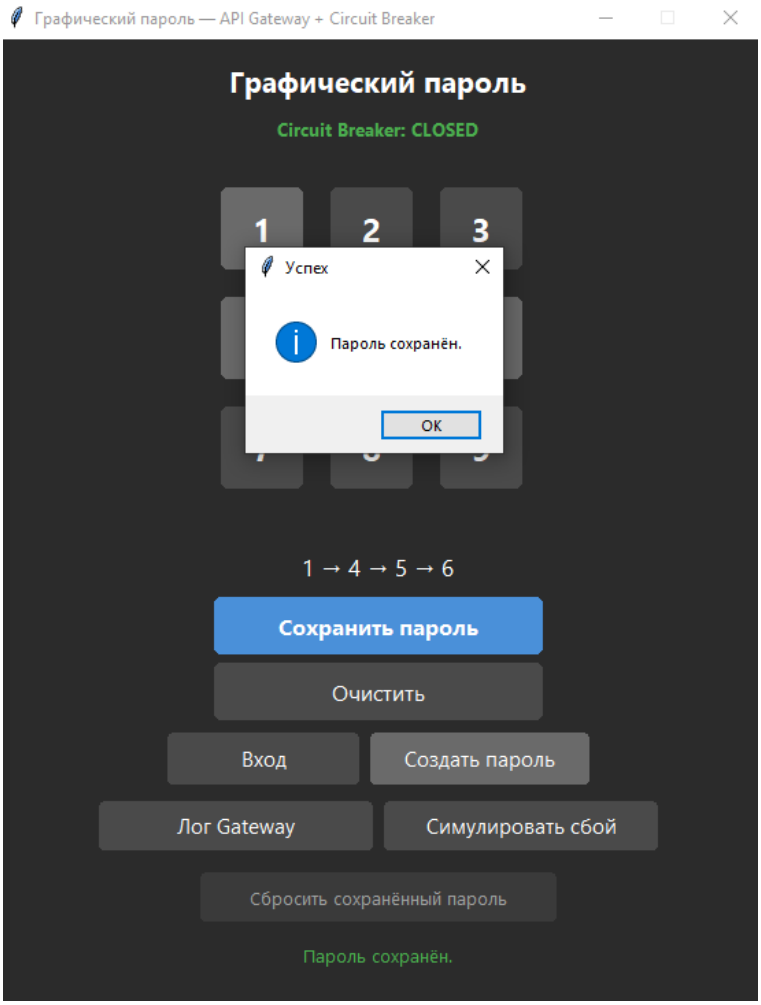


Рисунок 5 - Создание пароля.

В режиме «Создать пароль» нажата кнопка «Сохранить пароль». Появляется диалог «Пароль сохранён». В graphical_password.json записан SHA-256 хэш.

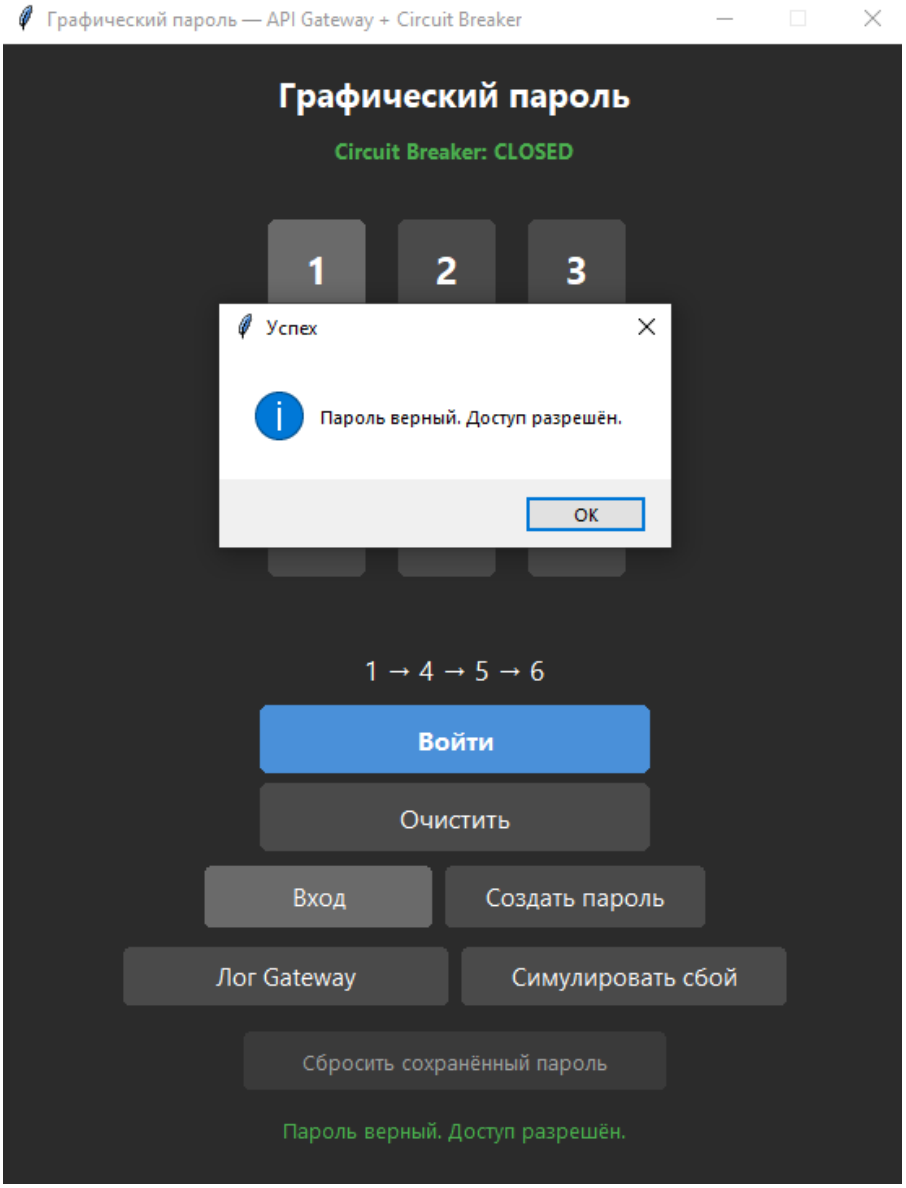


Рисунок 6 - Проверка правильного пароля.

В режиме «Вход» введена та же последовательность. Диалог: «Пароль верный. Доступ разрешён.».

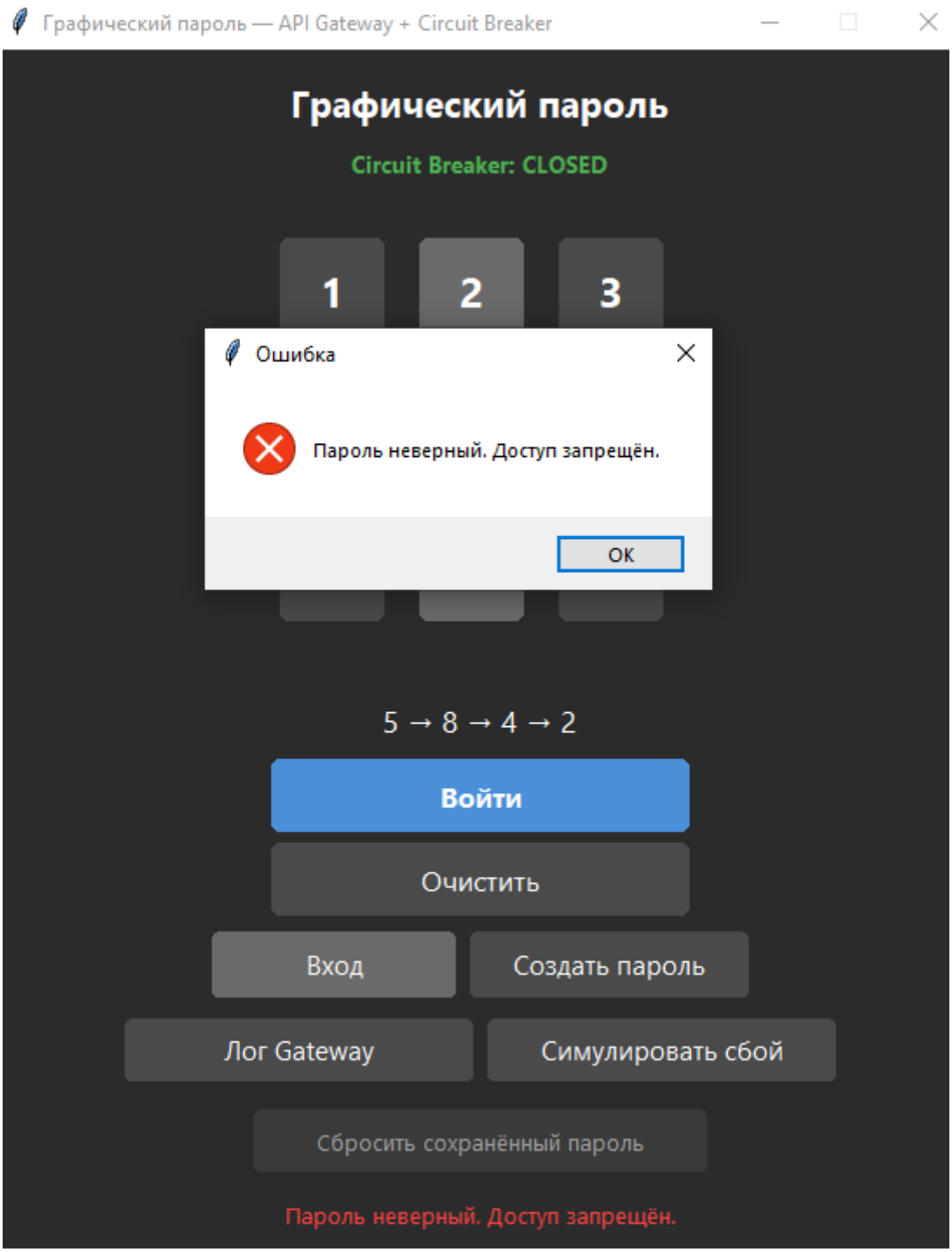


Рисунок 7 - Проверка неправильного пароля.

Введена 5 → 8 → 4 → 2. Диалог: «Пароль неверный. Доступ запрещён.».

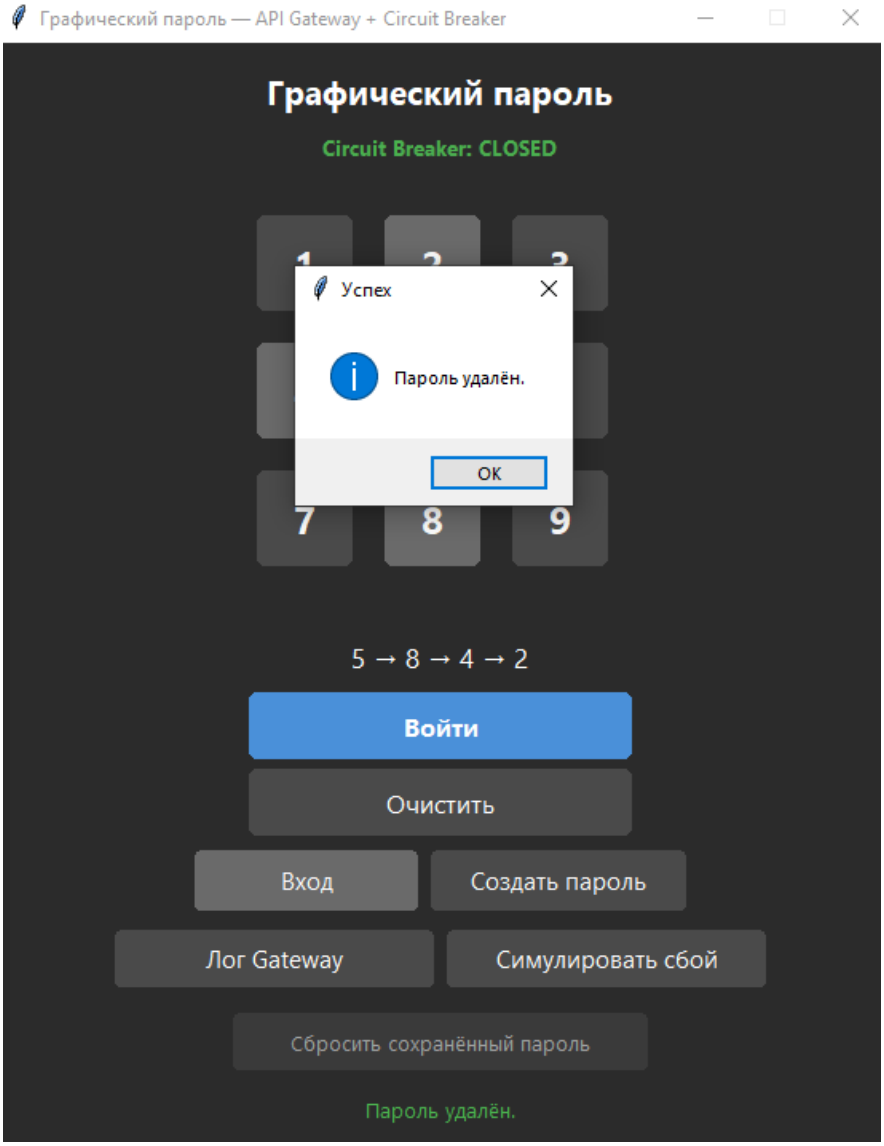


Рисунок 8 - Сброс сохранённого пароля.

Нажата кнопка «Сбросить сохранённый пароль». Диалог «Пароль удалён.», файл graphical_password.json удалён.

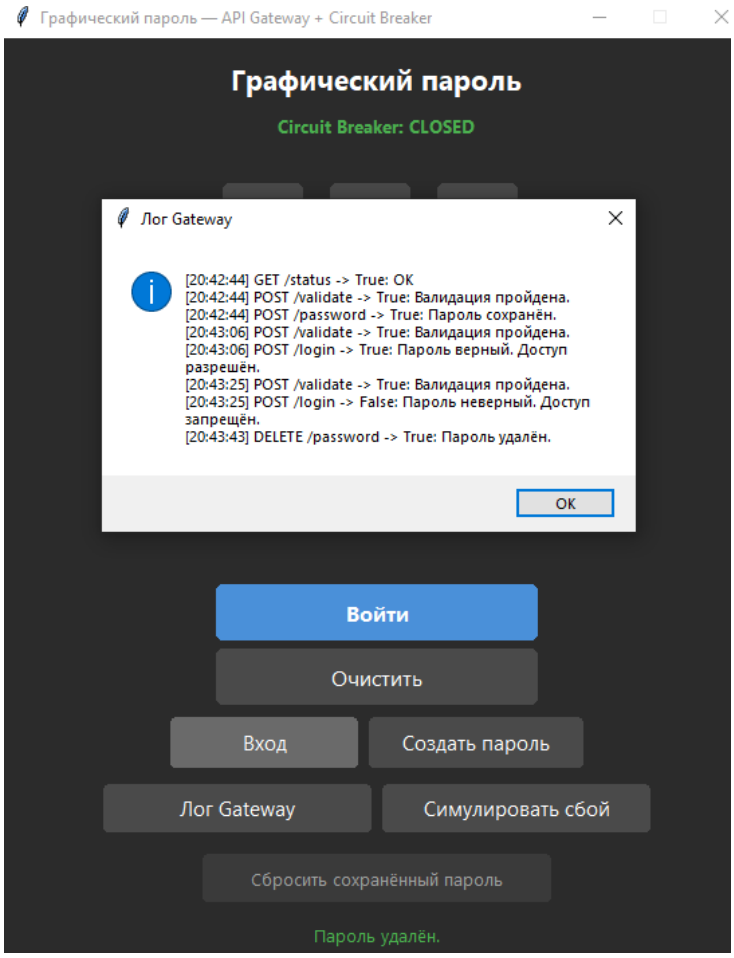


Рисунок 9 - Лог Gateway

Кнопка «Лог Gateway» показывает последние 15 записей вида.

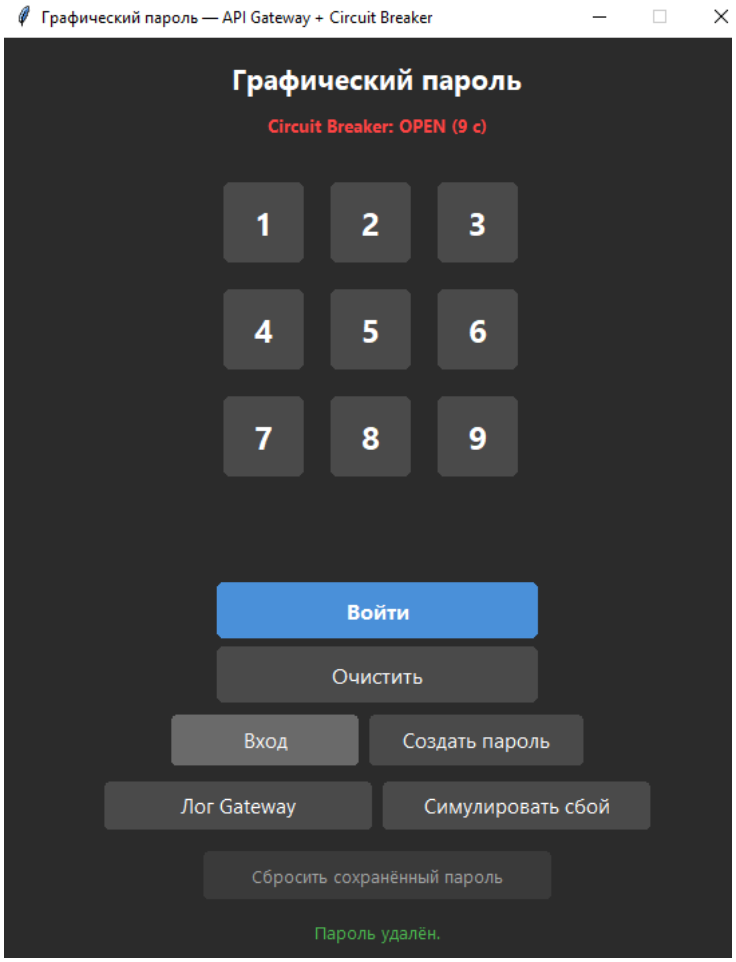


Рисунок 10 - Circuit Breaker в состоянии OPEN.

Три нажатия кнопки «Симулировать сбой» - бейдж становится красным: Circuit Breaker: OPEN (10 с). При попытке входа появляется сообщение «Circuit Breaker открыт. Подождите N с.». Через 10 секунд бейдж снова зелёный.

Вывод

В ходе выполнения лабораторной работы были изучены и реализованы два паттерна микросервисной архитектуры:

API Gateway - единая точка входа для клиента, скрывающая внутреннюю структуру «микросервисов» (PasswordService, ValidatorService). UI работает только с APIGateway, что позволяет менять состав и реализацию сервисов без правок клиента.

Circuit Breaker - предохранитель, который после 3 ошибок «размыкает цепь» на 10 секунд и блокирует запросы к сбойному сервису, предотвращая каскадные отказы.

Разработанное приложение «Графический пароль» демонстрирует совместное применение этих паттернов: шлюз маршрутизирует запросы и ведёт журнал, а предохранитель защищает систему от повторных вызовов при сбоях. Код разделён на слои (сервисы, шлюз, UI), что упрощает тестирование и развитие.